

INSTITUTO TECNOLÓGICO DE COSTA RICA
Campus Tecnológico Central Cartago
Escuela de Ingeniería en Computación

IC3101 – Arquitectura de Computadoras
II Semestre 2024

Reporte #9- Semana 10 – Sivarama - Capitulo 16
Operaciones lógicas y de bits

Estudiante: Mauricio González Prendas – **Carné:** 2024143009

Profesor: Esteban Arias Méndez

Fecha de entrega: 30/09/2024

Bibliografía:

[1] S. P. Dandamudi, *Guide to Assembly Language Programming in Linux*. Springer Science & Business Media, 2005, pp. 341–360.

Abstract—Logical and bit manipulation instructions in assembly language are crucial for handling low-level data operations. Assembly language provides five main logical instructions (and, or, not, xor, and test), which are used for manipulating bits in logical expressions and bitwise operations. Additionally, shift (shl, shr, sal, sar) and rotate (rol, ror, rcl, rcr) instructions enable efficient multiplication, division by powers of 2, and bit rearrangement. Bit manipulation instructions, such as bt, bts, btr, and btc, allow testing and modifying individual bits, while bit scan instructions (bsf, bsr) help locate specific bits within a register or memory. These operations are essential for tasks such as parity generation, encryption, and efficient data processing at the assembly level.

I. Introducción

Las operaciones lógicas y de manipulación de bits son esenciales en muchos lenguajes de alto nivel, y la programación en ensamblador ofrece una variedad de instrucciones específicas para este propósito. Estas instrucciones permiten manipular expresiones lógicas y realizar operaciones bit a bit. También se proporcionan instrucciones para el desplazamiento y la rotación de bits, facilitando la multiplicación y división por potencias de 2. Además, existen instrucciones para probar y modificar bits individuales. Este resumen cubre estas instrucciones con ejemplos de su aplicación en el contexto de la programación en ensamblador.

II. Instrucciones lógicas

El ensamblador proporciona cinco instrucciones lógicas principales: and, or, not, xor y test. Excepto not, todas son binarias, operan sobre operandos de 8, 16 o 32 bits, y afectan algunas banderas de estado del procesador. Las banderas afectadas incluyen la bandera de cero (ZF), la bandera de signo (SF) y la bandera de paridad

(PF), mientras que la bandera de acarreo (CF) y la bandera de desbordamiento (OF) se despejan.

1. **Instrucción and:** Utilizada para implementar expresiones lógicas, borrar bits o aislar bits específicos de un operando. Por ejemplo, para borrar los dos bits menos significativos de un registro, se puede aplicar una máscara de bits usando and.

Ejemplo:

AL = 11010110B

BL = 11111100B

and AL, BL ; Resultado: 11010100B

Este código limpia los dos bits menos significativos de AL.

2. **Instrucción or:** Se utiliza para implementar expresiones lógicas o establecer bits específicos de un operando. Por ejemplo, para forzar ciertos bits a 1 en una operación lógica.

Ejemplo:

AL = 11010110B

BL = 00000011B

or AL, BL ; Resultado: 11010111B

Aquí, los dos bits menos significativos se configuran en 1.

3. **Instrucción xor:** Se usa para alternar bits o para inicializar registros a cero.

Ejemplo:

`xor EAX, EAX ; Inicializa EAX a 0`

4. **Instrucción not:** Complementa todos los bits de un operando, invirtiendo cada bit (1 a 0 y viceversa).
5. **Instrucción test:** Realiza una operación and sin modificar el operando y solo actualiza las banderas, útil para pruebas no destructivas de bits.

III. Instrucciones de desplazamiento

Existen dos tipos de instrucciones de desplazamiento: lógico y aritmético. Estas instrucciones permiten desplazar bits a la izquierda o a la derecha, siendo útiles para manipulación de bits y operaciones matemáticas rápidas como la multiplicación y división por potencias de 2.

1. **Desplazamiento lógico (shl, shr):** Desplazan los bits hacia la izquierda (shl) o derecha (shr) e introducen ceros en los espacios vacíos. Útil para multiplicar o dividir números sin signo por potencias de 2.

Ejemplo:

```
shl EAX, 1    ; Multiplica el valor de EAX por 2
shr EAX, 1    ; Divide el valor de EAX por 2
```

2. **Desplazamiento aritmético (sal, sar):** Similar al desplazamiento lógico, pero preserva el bit de signo en el caso de sar, útil para números con signo.

Ejemplo:

```
sal EAX, 1    ; Multiplica número con signo por 2
sar EAX, 1    ; Divide número con signo por 2
```

IV. Instrucciones de rotación

Las instrucciones de rotación permiten rotar bits hacia la izquierda o derecha, manteniendo los bits desplazados. Hay dos tipos: con o sin la participación de la bandera de acarreo (CF).

1. **Rotación sin acarreo (rol, ror):** Rotan los bits sin implicar la bandera de acarreo.

Ejemplo:

```
rol AL, 4     ; Rota 4 bits a la izquierda
```

2. **Rotación con acarreo (rcl, rcr):** Rotan los bits y utilizan la bandera de acarreo como parte del proceso de rotación.

V. Instrucciones de manipulación de bits

El conjunto de instrucciones para manipulación de bits permite probar y modificar bits específicos de un operando.

1. **Instrucciones de prueba y modificación de bits (bt, bts, btr, btc):** Estas instrucciones prueban el estado de un bit y lo modifican (poner a 1, a 0, o complementarlo) dependiendo de la instrucción utilizada.

Ejemplo:

```
bts EAX, 3    ; Establece el bit 3 de EAX
btr EAX, 3    ; Restablece el bit 3 de EAX
btc EAX, 3    ; Invierte el bit 3 de EAX
```

2. **Instrucciones de escaneo de bits (bsf, bsr):** Escanean un operando en busca del primer bit con valor 1, desde adelante (bsf) o desde atrás (bsr).

Ejemplo:

```
bsf ECX, EAX  ; Encuentra el primer bit a 1 en EAX y almacena la posición en ECX
```